

บทที่ 4 :การออกแบบ สถาปัตยกรรมระบบ

แนะนำการออกแบบระบบ **01**

Repository Architecture **02**

Layered Architecture **03**

Client/Server Architecture **04**

Model-View-Controller (MVC) **05**

1. แนะนำการออกแบบระบบ

1.1 การออกแบบระบบ

การออกแบบระบบ (System Design) คือ การกำหนดสถาปัตยกรรม ส่วนประกอบ ส่วนต่อประสาน และลักษณะด้านอื่น ๆ ของระบบ ซึ่งเป็นการนำเอาข้อกำหนดความต้องการระบบ (SRS) มากำหนดโครงสร้างของระบบเพื่อนำไปเขียนโค้ดและทดสอบ ในขั้นถัดไป

❑ การออกแบบระบบ แบ่งออกเป็น

1) การออกแบบเชิงสถาปัตยกรรม (Architectural Design)

เรียกอีกอย่างว่า Top-Level Design เป็นการกำหนดโครงสร้างระบบในระดับบน

2) การออกแบบอย่างละเอียด (Detailed Design)

เรียกอีกอย่างว่า Implementation Design เป็นการออกแบบส่วนประกอบของซอฟต์แวร์อย่างละเอียด เพื่อเอื้อต่อการเขียนโค้ดมากที่สุด

1.2 การออกแบบเชิงสถาปัตยกรรม

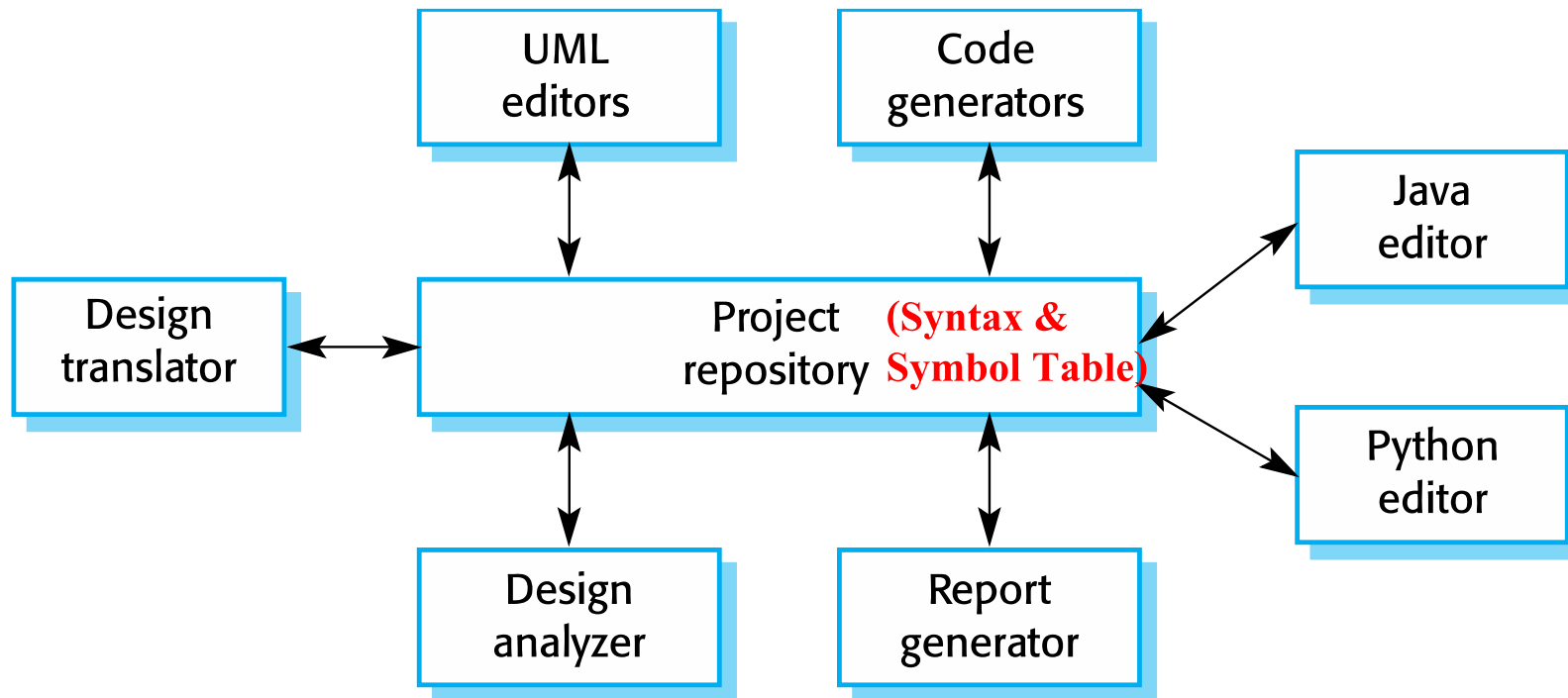
การออกแบบเชิงสถาปัตยกรรม (Architectural Design) คือ การกำหนดลักษณะโครงสร้างของระบบหรือซอฟต์แวร์ในมุมมองระดับบน ซึ่งแสดงให้เห็นถึงส่วนประกอบต่าง ๆ ของระบบ/ซอฟต์แวร์

❑ การแบบเชิงสถาปัตยกรรม ประกอบไปด้วย

- Repository Architecture
- Layered Architecture
- Client/Server Architecture
- Model-View-Controller (MVC)

2. Repository Architecture

- เป็นสถาปัตยกรรมที่เหมาะสมต่อการแชร์ข้อมูล (Data Sharing) ระหว่างระบบย่อย (Sub-system/Component)
- ข้อมูลทั้งหมดในระบบถูกจัดการที่ส่วนกลาง โดยที่ระบบย่อยไม่ได้ติดต่อกันเองโดยตรง



Integrated Development Environment (IDE) เป็นซอฟต์แวร์ที่รวมสิ่งจำเป็นต่อการพัฒนาซอฟต์แวร์ เช่น Editor, Compiler, Analyzer, Report Generator ฯลฯ ตัวอย่างของ IDE เช่น Android Studio, Xcode, VS Studio.NET

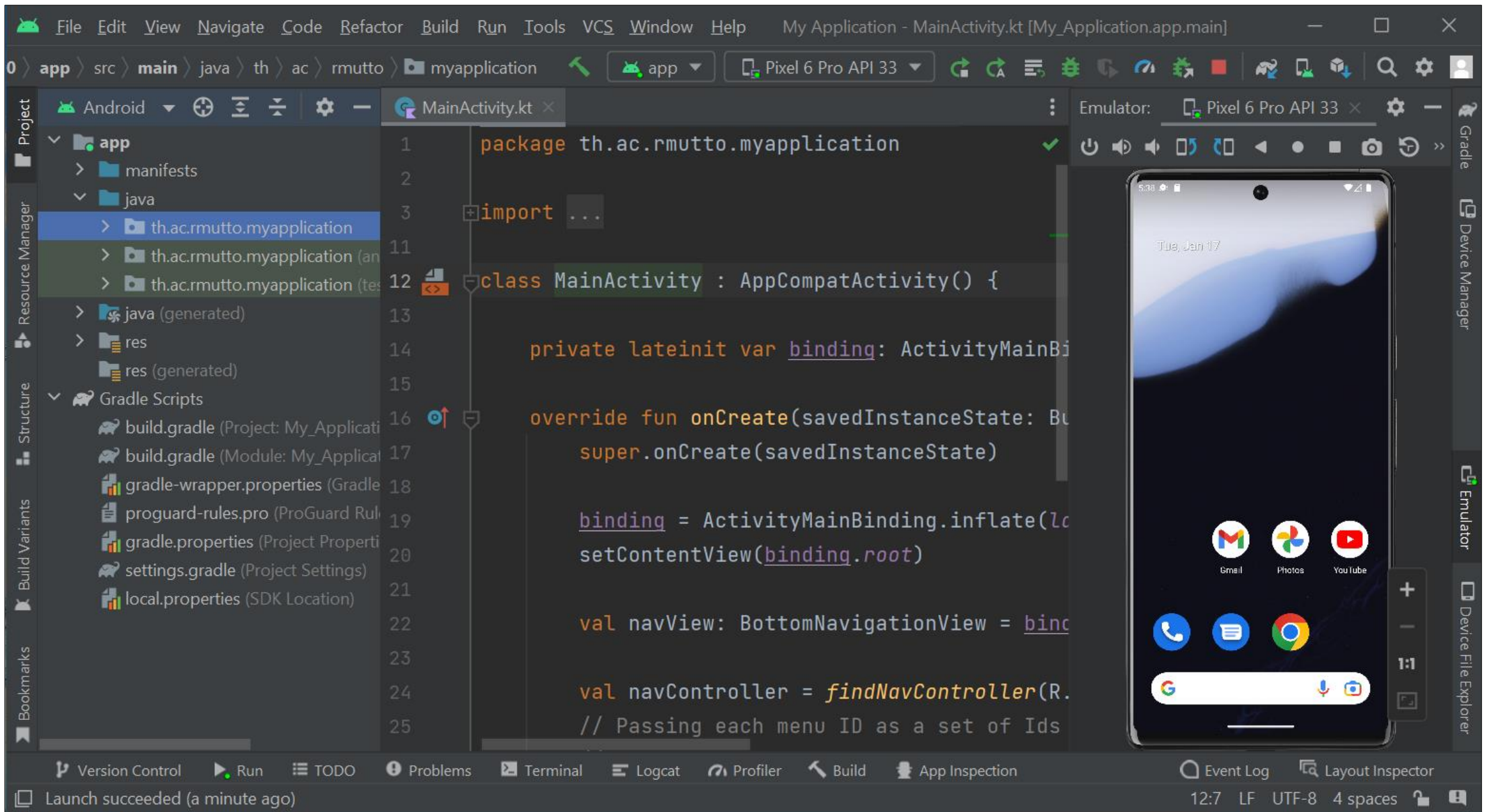
ภาพที่ 2-1 : Repository Architecture ของการ IDE

❑ ข้อดีของสถาปัตยกรรมแบบ Repository Architecture

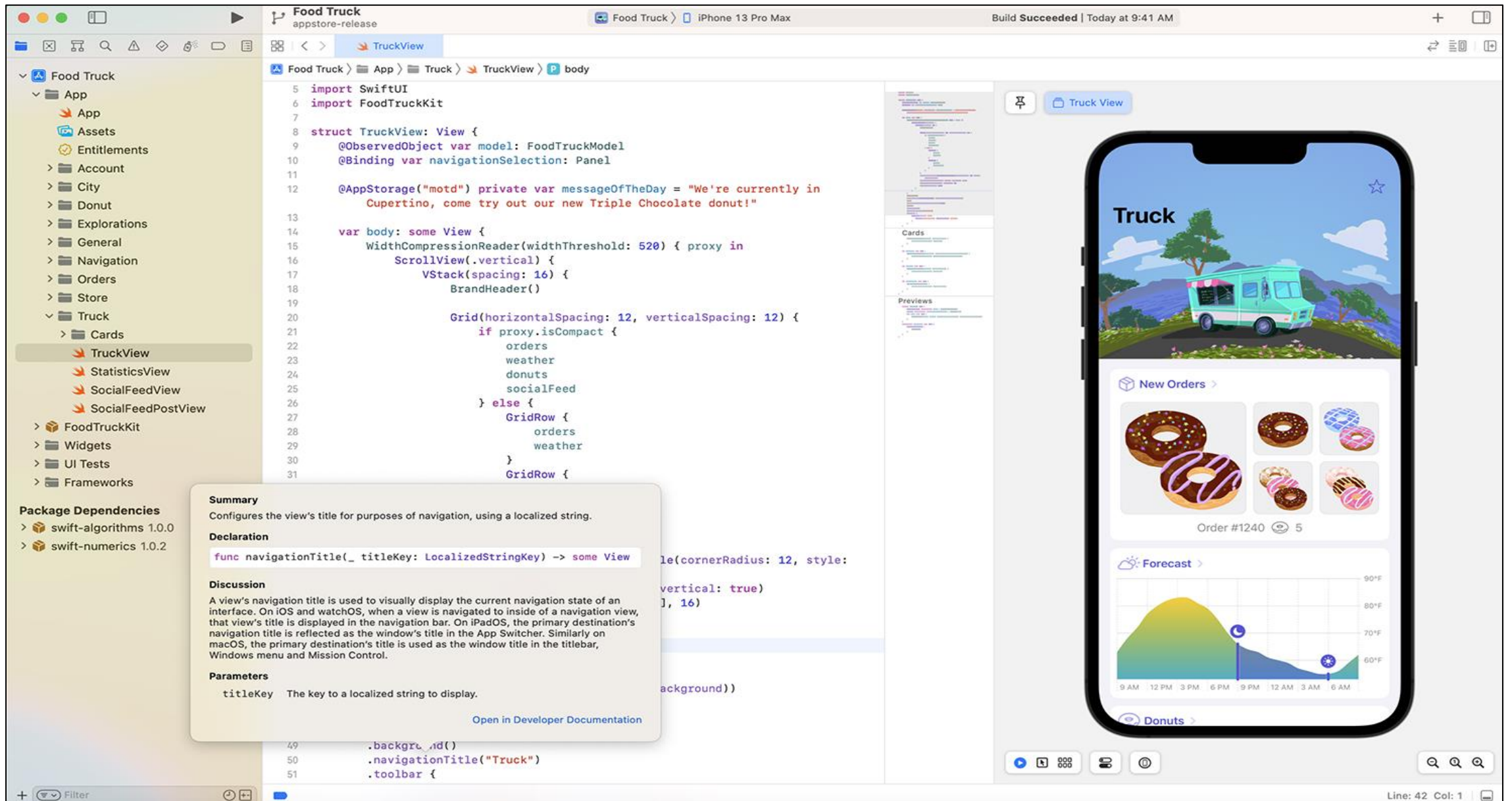
- ระบบย่อยมีความเป็นอิสระ ความผิดพลาดของระบบหนึ่งไม่ส่งผลกระทบต่อระบบอื่น
- สะดวกต่อการ Backup และการจัดการด้าน Security เนื่องจากดำเนินการที่ศูนย์กลาง

❑ ข้อเสียของสถาปัตยกรรมแบบ Repository Architecture

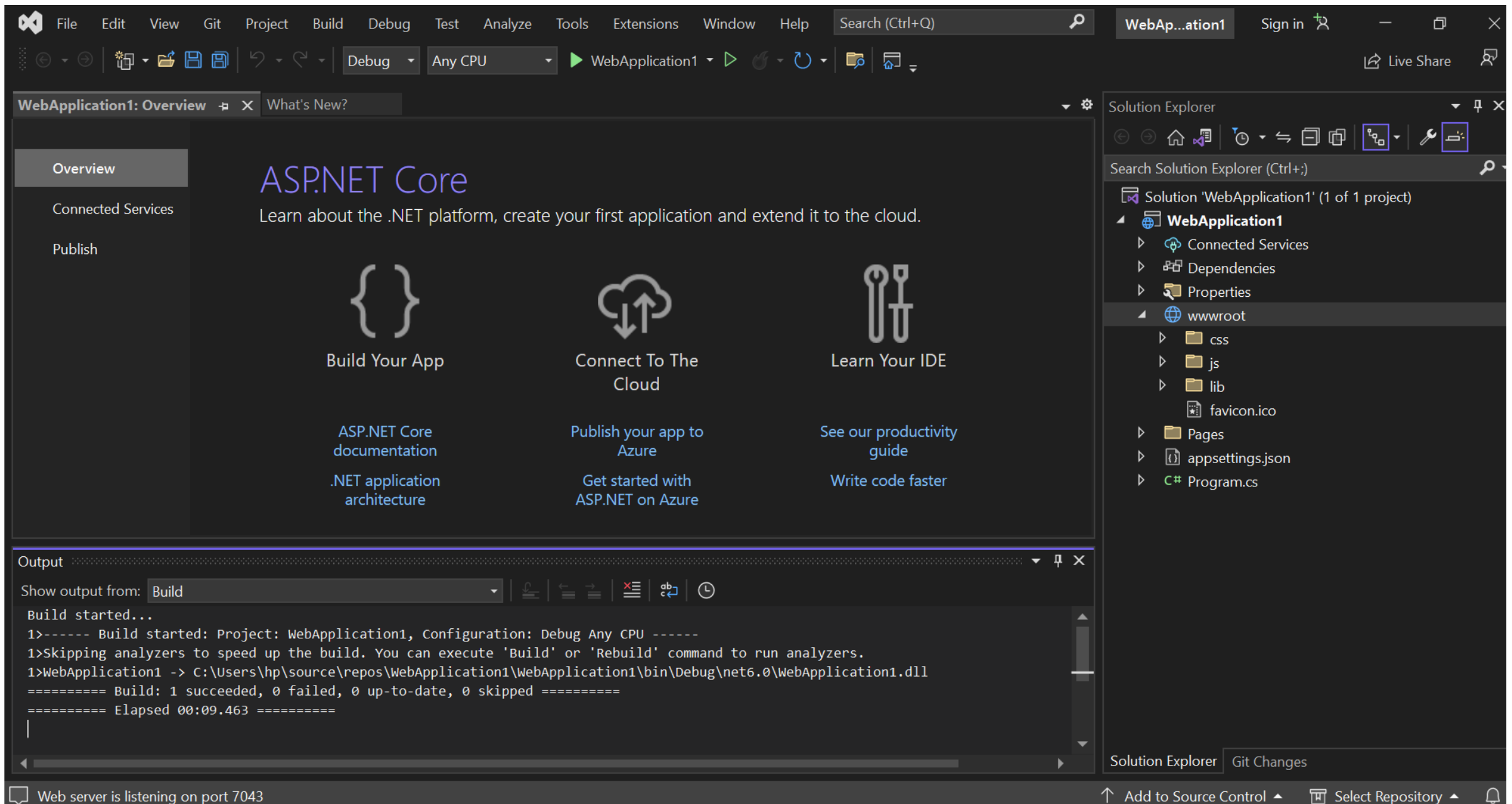
- เมื่อศูนย์กลางข้อมูลมีปัญหาจะส่งผลกระทบต่อทั้งระบบ
- อาจจะทำให้เกิดคอขวดหรือปัญหาในการสื่อสารจากทุกระบบย่อยผ่านศูนย์กลาง



ภาพที่ 2-2 : หน้าจอของ Android Studio



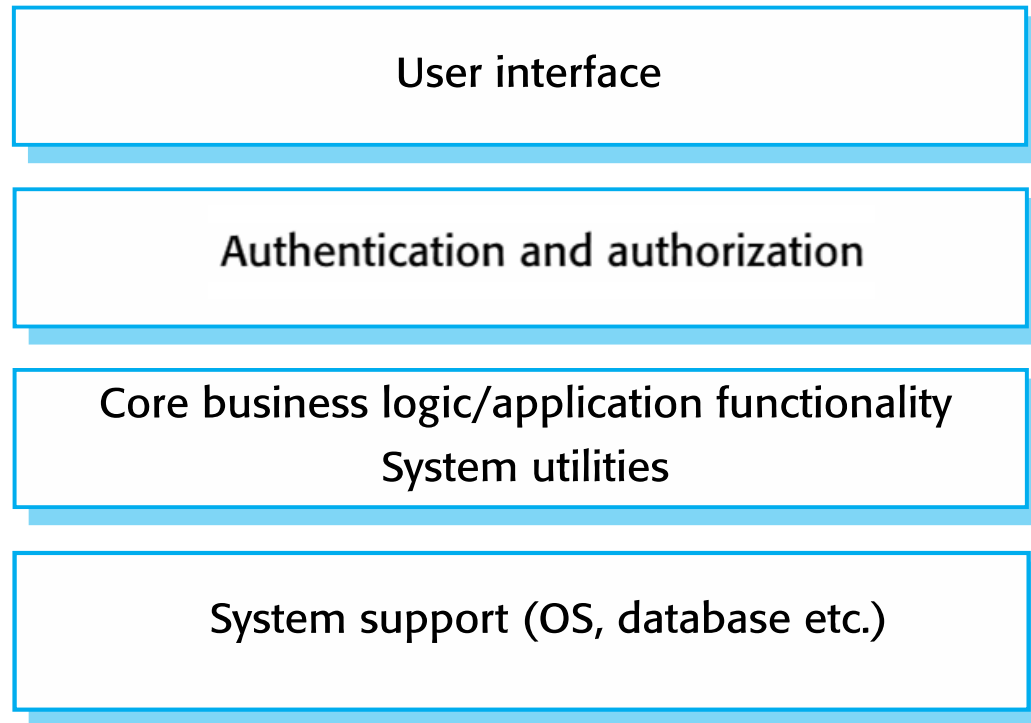
ภาพที่ 2-3 : หน้าจอของ Xcode



ภาพที่ 2-3 : หน้าจอของ Visual Studio .NET

3. Layered Architecture

- เป็นโครงสร้างที่แสดงให้เห็นการจัดโครงสร้างระบบย่อย ในมุมมองแบบระดับชั้น
- แต่ละชั้นจะทำหน้าที่ในแต่ละด้านของซอฟต์แวร์
- การทำงานมักจะติดต่อกันจากชั้นหนึ่งไปยังอีกชั้นหนึ่ง



ภาพที่ 2-5 : โครงสร้างแบบ Layer Architecture

❑ ข้อดีของสถาปัตยกรรมแบบ Layer Architecture

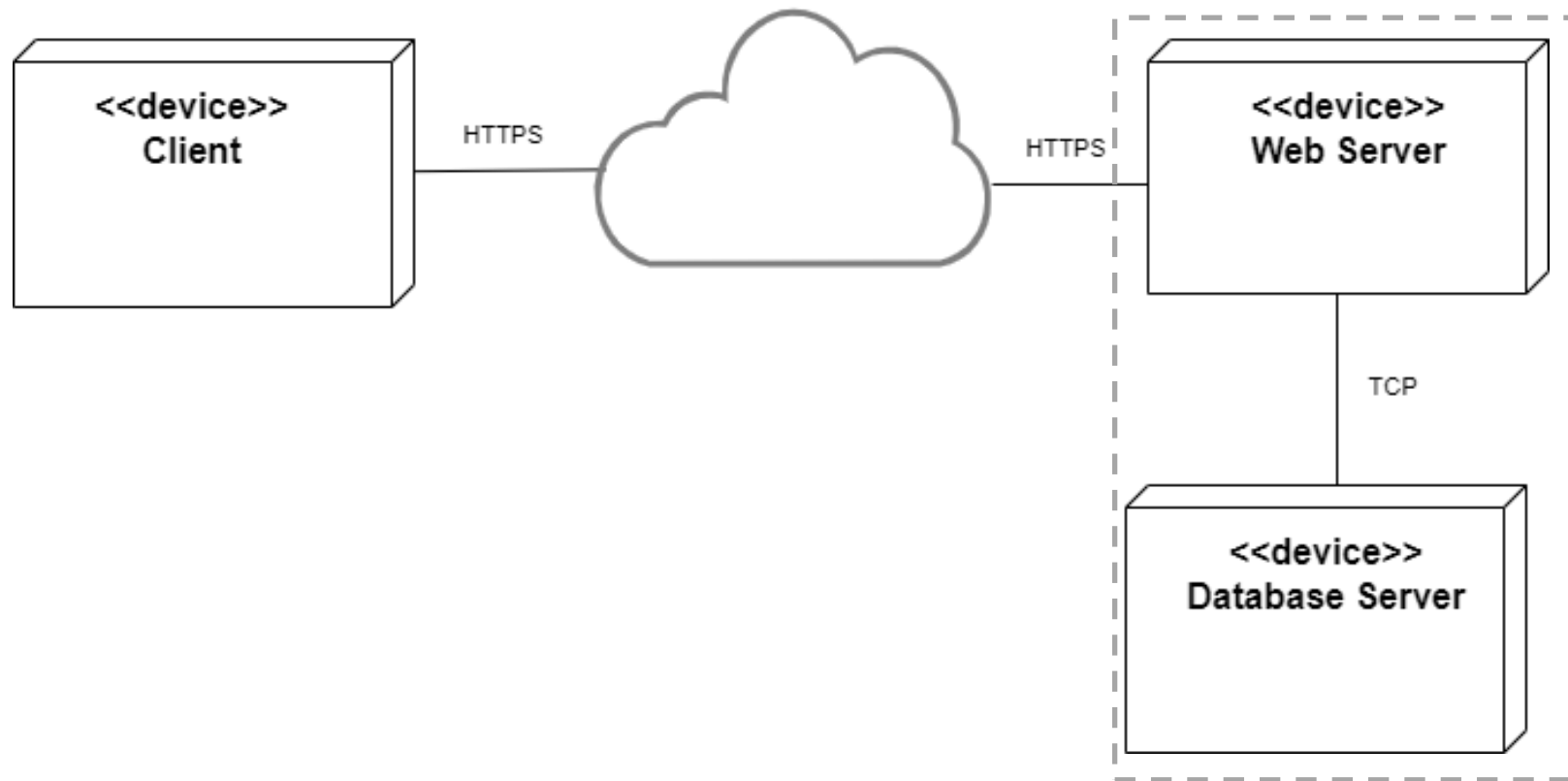
- สามารถเปลี่ยนแปลงแก้ไขแต่ละ Layer ได้สะดวก (ถ้าใช้มาตรฐานการเชื่อมต่อเดียวกัน)

❑ ข้อเสียของสถาปัตยกรรมแบบ Layer Architecture

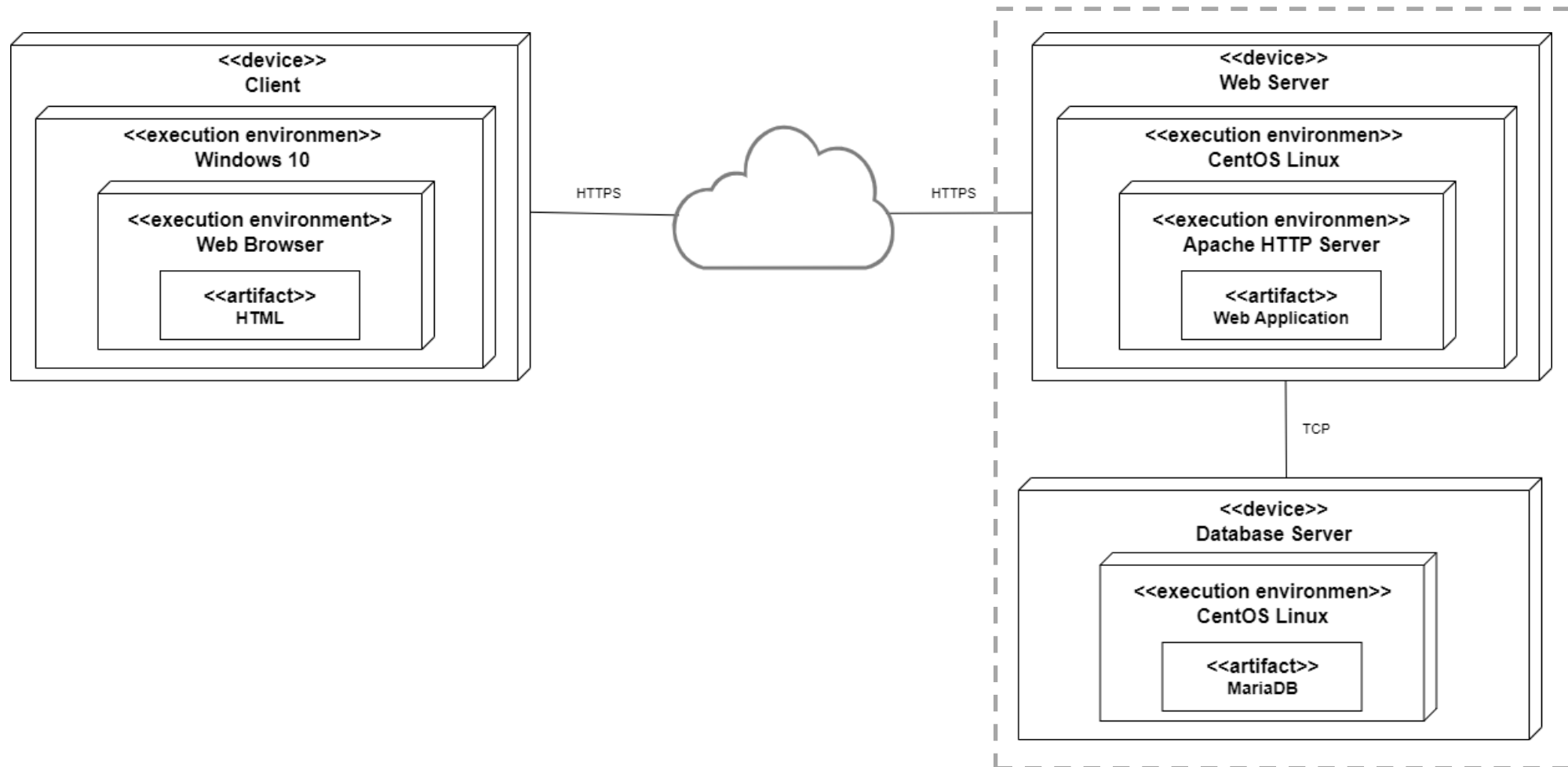
- ประสิทธิภาพอาจลดลง เนื่องจากต้องส่งคำสั่งผ่านหลายชั้น

4. Client/Server Architecture

- มีการสื่อสารกันระหว่าง Client กับ Server ผ่านทางเครือข่าย (Network)
- เป็นรูปแบบที่ได้รับความนิยมสูงสุดในปัจจุบัน



ภาพที่ 2-6 : Client/Server Architecture



ภาพที่ 2-7 : Client/Server Architecture (แบบละเอียด)

❑ องค์ประกอบของสถาปัตยกรรมแบบ Client/Server Architecture ประกอบด้วย 3 ส่วน

- **Server** คือ เครื่องคอมพิวเตอร์แม่ข่ายที่ให้บริการต่าง ๆ ต่อเครื่องคอมพิวเตอร์ลูกข่าย เช่น
 - Web Server : ให้บริการซอฟต์แวร์ต่าง ๆ เช่น ระบบซื้อขายสินค้าออนไลน์ ระบบทะเบียนมหาวิทยาลัย ระบบ e-banking ฯลฯ
 - Mail Server : ให้บริการอีเมล
 - Print Server : ให้บริการการพิมพ์เอกสาร
- **Client** คือ เครื่องคอมพิวเตอร์ลูกข่ายที่เรียกใช้บริการต่าง ๆ จากเครื่องแม่ข่าย ในการเรียกใช้บริการอาจต้องอาศัยซอฟต์แวร์ช่วย เช่น Web browser สำหรับบริการเว็บไซต์/เว็บแอปพลิเคชัน MS Outlook สำหรับบริการอีเมล
- **Network** คือ เครือข่ายคอมพิวเตอร์ที่เชื่อมโยงระหว่าง Server และ Client เช่น เครือข่ายอินเทอร์เน็ต (Internet) เครือข่ายอินทราเน็ต (Intranet)

❑ ข้อดีของสถาปัตยกรรมแบบ Client/Server Architecture

- ง่ายต่อการปรับปรุงระบบทั้งในส่วนของซอฟต์แวร์และฮาร์ดแวร์
- ง่ายต่อการจัดการกับข้อมูลและไฟล์ข้อมูล

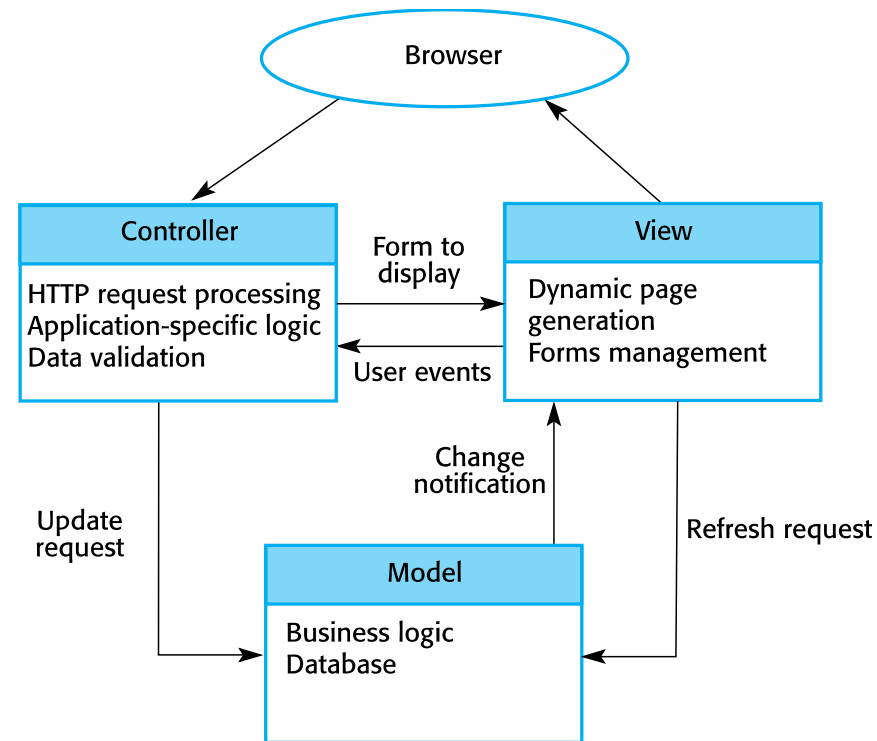
❑ ข้อเสียของสถาปัตยกรรมแบบ Client/Server Architecture

- ประสิทธิภาพของระบบส่วนหนึ่งขึ้นอยู่กับ Network ดังนั้นเมื่อ Network มีปัญหาผู้ใช้งานก็จะไม่สามารถใช้บริการได้
- แต่ละ Service ให้บริการ ณ จุดเดียว ซึ่งจะทำให้บริการหยุดชะงักทันที เมื่อ Server ที่ให้บริการถูกโจมตี

5. Model-View-Controller (MVC)

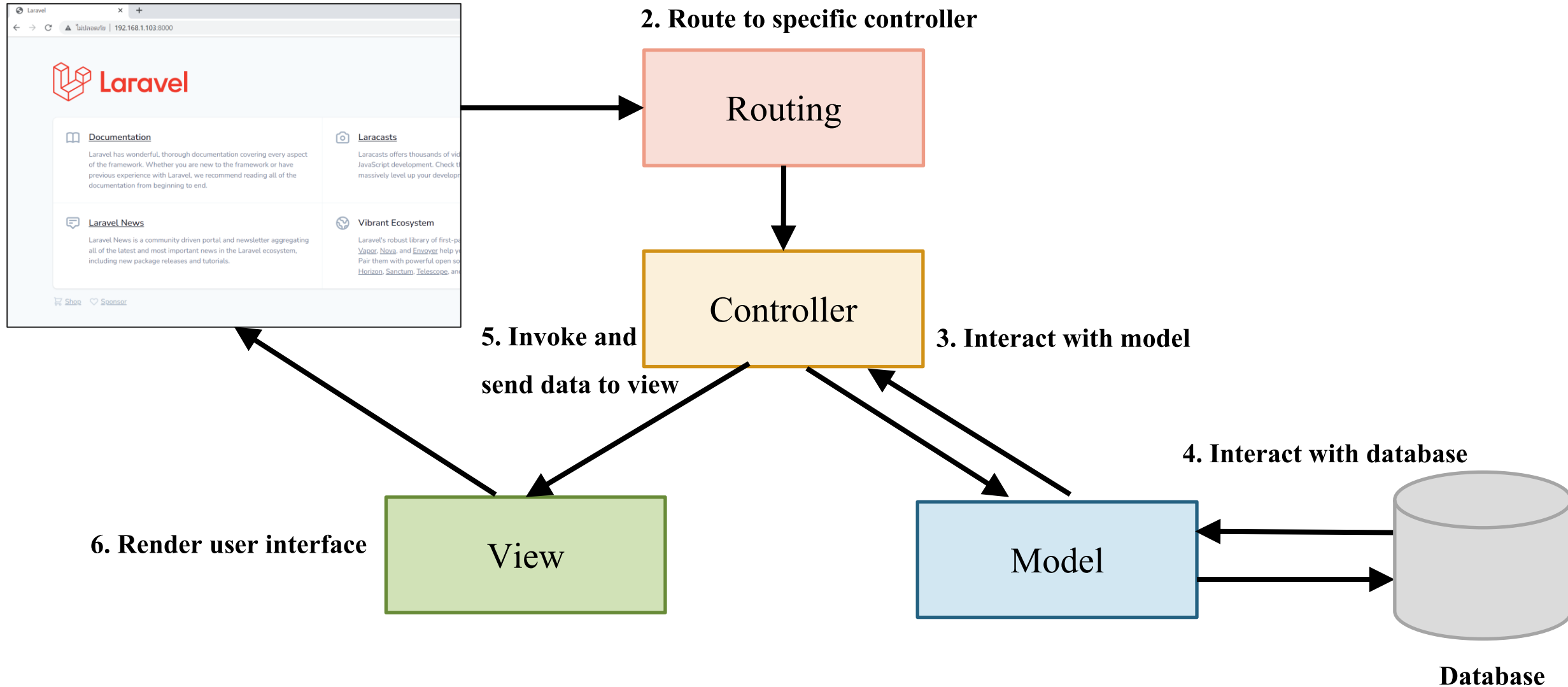
เป็นการออกแบบเชิงสถาปัตยกรรมที่แยกองค์ประกอบของแอปพลิเคชันออกเป็น 3 ส่วน ประกอบด้วย

- 1) **Model** : ส่วนที่ติดต่อกับฐานข้อมูล ,
- 2) **View** : ส่วนของการแสดงผล, และ
- 3) **Controller** : ส่วนของการควบคุม Request จาก User



ภาพที่ 2-8 : MVC Architecture

1. User Request



ภาพที่ 2-9 : MVC Architecture (Laravel)

Do you have any questions?

